

Decomposing Lexicon-Assisted Correction for Word-Level Handwritten Text Recognition on IAM: Lexicon Coverage, Frequency Ranking and Line Context

Rıdvan Dursun

*Department of Software Engineering, Faculty of Computer and Information Sciences,
Sakarya University, 54050 Sakarya, Türkiye
ridvan.dursun@ogr.sakarya.edu.tr*

Berhat Yeşilyurt

*Department of Software Engineering, Faculty of Computer and Information Sciences,
Sakarya University, 54050 Sakarya, Türkiye
berhat.yesilyurt@ogr.sakarya.edu.tr*

Nur Banu Oğur*

*Department of Software Engineering, Faculty of Computer and Information Sciences,
Sakarya University, 54050 Sakarya, Türkiye
nbogur@sakarya.edu.tr*

Received (Day Month Year)

Revised (Day Month Year)

Accepted (Day Month Year)

Handwriting varies strongly between writers, which makes word-level recognition hard. We study it on the IAM database under the Aachen writer-disjoint protocol, on all 20 310 test words. Our system, CRNN-LX, is a convolutional-recurrent network trained from scratch, followed by a lexicon-aware post-corrector that rescores candidates with a Kneser–Ney trigram, using the recognizer’s own output for the preceding words on the text line as context. Two questions are examined. The first concerns the post-corrector, usually reported as a single language-model step. Decomposing it shows that the lexicon dominates the ranking rule: a lexicon read off the language-model corpus (57 K types) beats a 239 K-word English list by 2.1 points although it is four times smaller, because it holds the proper nouns, inflections and capitalizations people write. Ranking adds about a point and line context almost as much, moving accuracy from 78.83% to 83.80% while lowering the character error rate; imposing the same lexicon inside the decoder changes little. The second question is whether writing-aware augmentation helps. Six configurations differing only in augmentation show no detectable gain from elastic, morphological and wide photometric transforms, whereas removing the conventional pipeline costs 2.24 points; over three seeds the transforms’ difference changes sign.

Keywords: Handwritten text recognition; lexicon-based post-correction; Kneser–Ney tri-

*Corresponding author.

gram; line context; convolutional-recurrent neural network; connectionist temporal classification; data augmentation; ablation study.

1. Introduction

Offline *handwritten text recognition* (HTR) is the task of converting a scanned image of handwriting into machine-readable text without access to the pen trajectory. Depending on how the input is segmented, an HTR system may operate on full pages, on text lines, or — as in this work — on *word* images. Word-level recognition is the harder setting per sample, because the optical model sees one crop at a time and cannot exploit the surrounding words to disambiguate an unclear glyph. When the crops are cut from text lines, as in IAM, that context is not lost entirely: the recognizer’s own outputs for the neighbouring crops are available, and whether a post-corrector can use them is one of the questions examined here.

The IAM Handwriting Database¹⁴ is the standard benchmark for English HTR, containing roughly 115 000 word images from 657 writers. We adopt the partition released by RWTH Aachen University¹⁹ — referred to as the *Aachen split* throughout this paper — which guarantees that no writer contributes to more than one of the training, validation and test sets. This *writer-disjoint* protocol is stricter than a random split and is the realistic condition for deploying a recognizer on handwriting it has never seen.

Modern recognizers are trained with the *connectionist temporal classification* (CTC) objective,⁷ which removes the need for character-level alignment between the image and its transcription. Two model families dominate: convolutional-recurrent networks (CRNNs) trained with CTC,^{1,17,21} and attention-based sequence-to-sequence or transformer encoders.^{10,15} For a compact CRNN, two ingredients are commonly credited with closing part of the gap to heavier models: data augmentation that reflects how handwriting actually varies, and a lexical prior applied at or after decoding. Both are widely used, but their individual contribution is rarely measured under controlled conditions — same data, same architecture, same training schedule, same test set.

This paper provides such a measurement, and finds the two ingredients unequal. On the augmentation side, the object of study is three writing-aware transforms added on top of a conventional geometric and photometric pipeline (rotation, translation, scaling, shear, brightness and contrast jitter, Gaussian noise, gamma correction and random erasing; Section 3.4): *elastic deformation*, which imitates the small variations of a stroke from one writing to the next; *morphological perturbation*, which thins or thickens the strokes as a fine or a broad pen would; and a *widened photometric range*, which imitates lighter or darker ink and scans. Throughout the paper we call these three the *proposed transforms* and everything else the *conventional pipeline*. The label is ours rather than a standard: there is no agreed set of routine augmentations for handwriting, and some prior systems already treat elastic distortion as one.⁵ On the lexical side, the post-corrector replaces a recognized

word by a lexicon entry within a small edit distance and ranks the candidates with a word-level language model; in the proposed system that model is a Kneser–Ney trigram conditioned on the recognizer’s own output for the preceding crops of the same text line. Which words the lexicon contains turns out to matter more than how the candidates are ranked, so we compare three lexicons and, for each, four ways of choosing inside it. We name the proposed system CRNN-LX (CRNN with lexicon-assisted, language-model-ranked post-correction) and its lexicon-free counterpart CRNN-G; Table 1 defines the six configurations compared. Our contributions are:

- **An anatomy of lexicon-assisted correction.** Holding the optical model fixed, we vary the lexicon, the rule that ranks the candidates inside it, the corpus the rule is estimated on, the context it is given and whether the lexicon is imposed during decoding or afterwards. The lexicon dominates: read off the language-model corpus (57 382 types, 96.4% coverage of the test tokens) it is worth 2.14 pp over a 239 126-type English word list (93.9%) under the same corrector, although it is four times smaller. The ranking rule adds 1.19 pp, the trigram with line context a further 0.89 pp, and word beam search — the same lexicon imposed inside the decoder — ends 0.32 pp away ($p = 0.014$), with a worse character error rate. We also find that what makes a small lexicon harmful is not its size but the obligation to replace: the 7 K training vocabulary costs 1.79 pp when every unlisted hypothesis must be replaced and *gains* 2.49 pp when the decoder may keep it. The complete corrector moves CRNN-G from 78.83% to 83.80%.
- **A controlled augmentation ablation with a positive control.** Six networks that differ only in the augmentation schedule are trained from scratch on the full Aachen training set and evaluated on the full test set. Elastic deformation, morphological perturbation and a widened photometric range change word accuracy by no more than 0.48 pp, and none of those differences reaches the significance threshold ($p < 0.01$) used throughout; removing the conventional pipeline underneath them costs 2.24 pp at $p = 1 \times 10^{-22}$. Repeating the two endpoint configurations with three seeds each shows why: the difference between them changes sign across seeds (-0.45 to $+0.66$ pp); all three per-seed tests fall below the conventional 0.05 level and they disagree in sign, while the mean difference is $+0.20 \pm 0.58$ pp. These results are consistent with a saturation effect rather than with an inability of the experimental setup to detect augmentation-related improvements.
- **A methodological observation on elastic deformation.** The common parameterization — normalized Gaussian-smoothed noise scaled by $\alpha \in [2, 5]$ — yields root-mean-square (RMS) displacements of a few hundredths of a pixel per axis at the working resolution, never more than 0.19 px anywhere, and is effectively a no-op; we report the corrected parameterization we used and its effect.
- **A verified evaluation protocol and full release.** Writer-, prompt- and form-level disjointness of the split are checked programmatically; all 20 310 test words are used; code, split lists and per-word predictions are public.

Scope of the claim. The claims of this paper are about measurement, not about rank. Our optical model is a plain CRNN trained from scratch on the 48 K Aachen training words, with no synthetic pre-training, no transfer from scene text and no ensembling. With the lexical corrector it reaches 83.80% on the Aachen test set, which is 1.3pp above the 2018 attention baseline and 0.3–0.8pp below the two strongest published word-level systems, which use a language model or scene-text pre-training (Table 6). What the paper contributes is the anatomy of two components those systems rely on as well, measured where each can be isolated. The lexical corrector is taken apart into the lexicon, the ranking rule and the place it is applied, and those three effects differ by an order of magnitude. The augmentation ablation has a positive control and three seeds, and shows the effect under study to be smaller than the variation between two runs of the same configuration. Finally, a parameterization in common use turns out to leave the image unchanged. None of the three depends on our particular network, and each is reported so that it can be re-derived from the released per-word predictions.

Section 2 divides the related literature into four groups and positions our work within them. Section 3 describes the pipeline of Fig. 1. Section 4 formalizes the evaluation metrics and statistical tests. Section 5 reports the results, Section 6 analyzes them, and Section 7 concludes.

2. Related Work

We organize prior work into four groups according to how each relates to ours: (i) *recurrent CTC recognizers*, which share our optical backbone but are almost always evaluated on text lines; (ii) *attention and transformer recognizers*, which target the same word-level task as ours with a heavier decoding paradigm; (iii) *other word-level recognizers on IAM*, whose evaluation protocols differ from ours; and (iv) *lexical decoders*, which use a lexicon or language model inside the decoding loop, whereas our post-corrector applies it afterwards. For each group we state what we adopt and where we differ.

Recurrent CTC recognizers. Graves and Schmidhuber⁸ introduced multi-dimensional long short-term memory (LSTM) networks with CTC as an early end-to-end offline recognizer. Puigcerver¹⁷ showed that a one-dimensional bidirectional LSTM (BiLSTM) stacked on a convolutional neural network (CNN) feature extractor matches multi-dimensional recurrence at a fraction of the cost, defining the canonical CRNN baseline that our architecture follows. Bluche and Messina¹ added gated convolutions, and Flor et al.⁴ combined a CNN with a bidirectional gated recurrent unit (BGRU) and a language-model pipeline. Wigington et al.²⁴ studied augmentation for CNN-LSTM recognizers and reported gains from combined transforms on IAM lines. We adopt the CRNN backbone unchanged; our contribution lies in measuring, rather than assuming, the effect of the augmentation and of the lexical prior.

Attention and transformer recognizers. Sueiras et al.²³ and Kang et al.¹⁰ introduced sequence-to-sequence recognizers with content-based attention for isolated IAM words. Kang et al.⁹ later integrated a language model into the decoder itself through candidate fusion, and Kass and Vats¹¹ combined a residual-network (ResNet) encoder, attention decoding and transfer learning from scene-text corpora, reporting the strongest word-level accuracy on the Aachen split that we are aware of. These four systems share our task, metric and split and are listed in Section 5. Transformer architectures such as TrOCR¹³ have since lowered line-level error rates further, but require pre-training on hundreds of millions of synthetic lines. Multi-modal large language models have also been benchmarked on IAM;³ those systems operate on full pages and rely on proprietary pre-training corpora, so they are not comparable to a from-scratch word recognizer.

Other word-level recognizers on IAM. Dutta et al.⁵ improved a CNN–recurrent neural network (RNN) hybrid with synthetic pre-training, slant normalization and test-time augmentation and report a 12.61% unconstrained word error rate on IAM words evaluated without punctuation or letter case, a setting not on the same footing as a case-sensitive recognizer trained from scratch. Rajesh et al.¹⁸ re-trained that CNN–RNN together with their own HWRCNet, a CNN–BiLSTM operating in the JPEG discrete cosine transform (DCT) domain, on a random 95:5 partition of IAM and obtained 77.14% and 80.08% strict word accuracy on uncompressed images; we note that their headline figure of 89.05% is *word accuracy with flexibility* of two edit operations rather than strict accuracy. Taking a different route altogether, Mondal et al.¹⁶ recognize IAM words lexicon-free by detecting individual characters with a YOLOv3 object detector, trained on only 1 200 word images, and report a 29.21% word error rate on 30 000 randomly selected test words. All three systems use partitions of IAM other than ours, a point we return to below.

Lexical decoders. Most systems that exploit a vocabulary do so while decoding. Word beam search (WBS)²⁰ injects a dictionary constraint directly into CTC beam search, so that only lexicon words can be produced; candidate fusion⁹ feeds a language model’s proposals into an attention decoder; and Flor et al.⁴ couple the optical model to a language-model pipeline. In all of these the lexical knowledge is part of decoding and cannot be varied without re-decoding or retraining. We instead apply it *after* greedy decoding, which lets us swap the lexicon, the language model, its corpus and its context independently while the optical hypotheses stay fixed. That is the property on which the decomposition of Section 5.2 rests. The language model itself is standard: an interpolated Kneser–Ney n -gram,^{2,12} the usual smoothing for word n -gram models. What differs is where its context comes from: a line-level decoder reads the preceding words off the same image, whereas our corrector reads them off the recognizer’s own outputs for the preceding word crops.

Positioning. Of the word-level systems above, only the attention-based recognizers of Sueiras et al.,²³ Kang et al.^{9,10} and Kass and Vats¹¹ are evaluated on the Aachen writer-disjoint partition that we use; the CRNN-family results of Rajesh et al.¹⁸ and the detector of Mondal et al.¹⁶ use their own random partitions of IAM. Comparisons across protocols are indicative only; the controlled comparisons of this paper are those between our own configurations.

3. Proposed System

Fig. 1 shows the complete processing chain, from the raw IAM word crops to the reported metrics, and marks the two stages whose effect this paper measures.

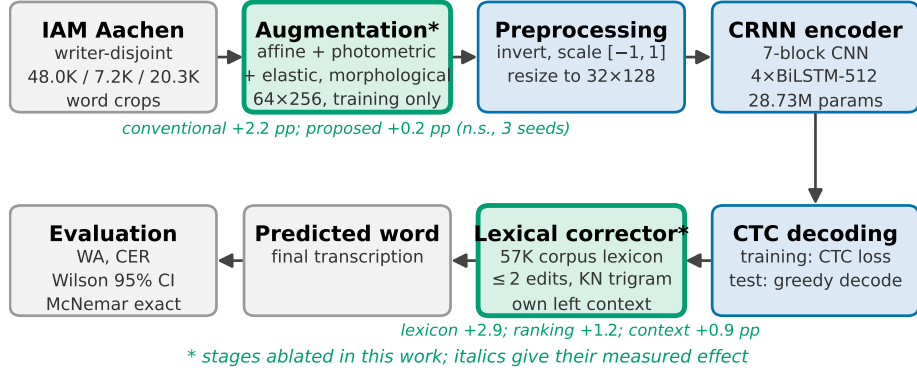


Fig. 1. Overview of CRNN-LX. Word crops are augmented (training only), normalized, encoded by a CRNN and decoded with CTC; the decoded string is then corrected against a 57K-type lexicon read off the language-model corpus, the candidates being ranked by a Kneser–Ney trigram conditioned on the system’s own output for the preceding words of the line. The two outlined stages are the ones varied in the ablations, and each carries the effects measured for it. The evaluation stage reports word accuracy (WA), character error rate (CER) and Wilson confidence intervals (CI), all defined formally in Section 4.

3.1. Data and Protocol

We use the word-level IAM partition of the Aachen split:¹⁹ 747 training forms, 116 validation forms and 336 test forms with disjoint writer sets (283, 56 and 161 writers respectively). Following common practice, only word crops whose segmentation is flagged ok are retained. Five validation forms whose prompt text also occurs in the test set were removed from the validation partition so that checkpoint selection is independent of the test prompts; each of them is the second half of a prompt whose first half is a test form, written by a different writer, so what overlaps is the text

and not the hand. IAM writes 57 of its prompts in two halves, and these five are the only ones whose halves fall in different partitions; none of them involves the training partition, so no prompt is shared between training and test. The test partition is used exactly as published. This yields 47 997 training words (two IAM image files are empty and are skipped), 7 205 validation words from 111 forms and 55 writers, and 20 310 test words from all 336 test forms. Writer, prompt and form disjointness between the three partitions, the counts just given, and the integrity of every record and image are verified by `cloud/verify_split.py`, distributed with the code. The label alphabet contains 78 symbols (upper- and lower-case Latin letters, digits and punctuation), plus the CTC blank.

3.2. Model Naming

Table 1 lists every configuration evaluated. All six share the optical model of Section 3.3, the training schedule and the post-corrector of Section 3.5; they differ only in the augmentation applied during training. CRNN-G is the fully augmented optical model decoded greedily, without any lexicon; CRNN-LX is that same model followed by the corpus-lexicon post-corrector of Section 3.5, whose candidates are ranked by the trigram with the system’s own left context, and is the configuration we propose. The suffixes denote the post-processing rather than the augmentation, which turns out not to matter (Section 5.1). The first row is the zero-augmentation anchor: it has the conventional pipeline switched off as well, and exists so that the ablation can distinguish “the proposed transforms add nothing” from “this pipeline cannot detect augmentation at all”.

Table 1. Configurations compared. All use the same 28.73 M-parameter network, the same schedule and the same post-corrector. The rows are cumulative: elastic and morphological perturbation are each added on top of the widened photometric range. Conventional: the pipeline listed in Section 3.4.

Name	Conventional	Wide photometric	Elastic	Morphological
no augmentation	no	no	no	no
CRNN-B (baseline)	yes	no	no	no
+ wide photometric	yes	yes	no	no
+ elastic	yes	yes	yes	no
+ morphological	yes	yes	no	yes
CRNN-LX (proposed)	yes	yes	yes	yes

3.3. Optical Model

The encoder is a seven-block convolutional feature extractor that collapses the input to a single row of 512-dimensional frames, followed by a four-layer bidirectional LSTM with 512 hidden units and a linear projection onto the 79 output classes; the network has 28.73 M parameters. Inputs are grayscale crops resized to 32×128 ,

inverted so that ink is high-valued, and scaled to $[-1, 1]$. The network is trained with the CTC objective⁷ using AdamW with learning rate $\eta = 7 \times 10^{-4}$ and weight decay 10^{-5} , five warm-up epochs followed by cosine decay, mixed precision, and batches of 128, for at most 100 epochs. Training stops when neither the validation loss nor the validation word accuracy has improved for 15 epochs, and the checkpoint with the best validation word accuracy is evaluated once on the test set. The validation accuracy used for early stopping and checkpoint selection is computed with the word-list lexicon and the unigram prior of Section 3.5, identically for all six configurations; the trigram corrector is applied only afterwards, to the frozen checkpoints, so that it never influences which weights are selected.

3.4. *Augmentation*

The baseline (CRNN-B) uses what we call the conventional pipeline. Each transform draws its own parameters and its own apply-or-skip decision per sample, with the probability given in brackets: rotation ($\pm 7^\circ$, [0.6]), translation ($\pm 5\%$, [0.6]), scaling (0.9–1.1, [0.3]), shear ($\pm 5^\circ$, [0.3]), brightness and contrast jitter (0.85–1.15, [0.6]), Gaussian noise ($\sigma = 0.03$ of the $[0, 1]$ intensity range, [0.4]), gamma correction (0.80–1.20, [0.4]) and random erasing of a patch of up to one sixth of each side ([0.3]). The proposed transforms, which the ablation variants add cumulatively, keep those probabilities and are:

- **Wide photometric range.** The same three transforms at the same probabilities over wider ranges: brightness and contrast 0.70–1.35, gamma 0.70–1.30, noise $\sigma = 0.05$.
- **Elastic deformation** (probability 0.5). A random displacement field is smoothed with a Gaussian kernel whose width is 8% of the larger image side, rescaled to unit RMS, and multiplied by an amplitude drawn uniformly from $[1, 3]$ pixels; the crop is warped accordingly. The unit-RMS normalization is essential: the frequently used variant that multiplies the *unnormalized* blurred noise by $\alpha \in [2, 5]$ ²² produces, at the 64×256 working resolution, a per-axis RMS displacement whose median is 0.015 px at $\alpha = 2$ and 0.038 px at $\alpha = 5$, and which never exceeds 0.19 px anywhere in the field, i.e. it leaves the image visually and numerically unchanged. Our own earlier implementation had this property.
- **Morphological perturbation** (probability 0.3, with a kernel side drawn uniformly from $\{1, 2\}$). With a 2×2 kernel the crop is eroded or dilated with equal probability, reproducing the difference between a fine and a broad nib; a 1×1 kernel leaves it unchanged, so the rate at which the crop is actually perturbed is 0.15.

Fig. 2 illustrates the transforms. All augmentation is applied on the graphics processing unit (GPU) to whole batches at a working resolution of 64×256 pixels (the median IAM word height is 68 px), with independent random parameters per sample, before the final resize to 32×128 . All six configurations take this same path:

the anchor differs from the other five only in that every transform is switched off, so its crops are resized, inverted and normalized exactly as theirs are and nothing else is done to them. Its deficit in Table 2 therefore cannot be an artefact of a different resampling chain.

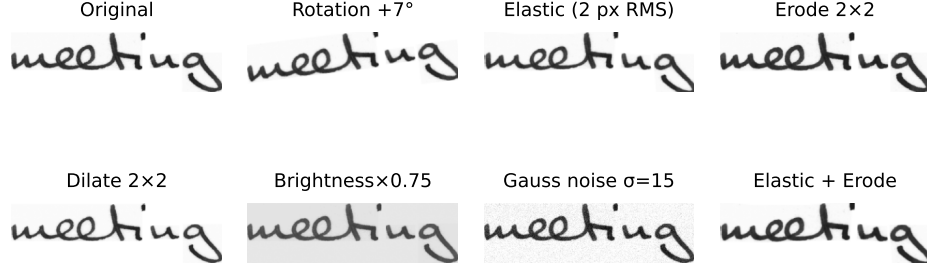


Fig. 2. Augmentation transforms on a training crop. The elastic panel uses the corrected amplitude (2 px RMS); the last panel shows the compound elastic+morphological effect. The parameter values are illustrative ones from within the ranges of Section 3.4; the noise panel is labelled in grey levels ($\sigma = 15$ of 255), which is the $\sigma = 0.05$ of Section 3.4 on the normalized scale.

3.5. Lexical Post-Correction: Lexicon, Ranking and Context

After greedy CTC decoding, each hypothesis is checked against a closed lexicon. If the hypothesis (case-insensitively) is in the lexicon it is accepted unchanged; otherwise it is replaced by the best-scoring lexicon entry within a Levenshtein distance of 1 (words of up to four characters) or 2 (longer words). On the test set 3084 hypotheses (15.2%) fall outside the corpus lexicon of the proposed system, and 2724 of them have at least one candidate within that distance; the other words pass through unchanged, so everything the corrector can gain or lose is decided among those 2724. A candidate c at edit distance d from the hypothesis is scored as

$$s(c) = \log P(c \mid h_1, h_2) - \alpha d, \quad (1)$$

where P is a word-level language model, $h_1 h_2$ are the system’s own outputs for the two crops that precede the hypothesis on its text line, and α is an edit penalty selected on the validation partition. The lexicon, the language model and the way the line is decoded are the variables of interest, and Section 5.2 varies them one at a time with the optical model held fixed.

Lexicon. Three lexicons are compared, and the difference between them turns out to matter more than anything else in the corrector:

- the *training lexicon*: the 7173 word types of the IAM training transcriptions;

- the *word-list lexicon*: those types plus the Natural Language Toolkit (NLTK) English word list, 239 126 types — a dictionary in the everyday sense, large but written without reference to any text;
- the *corpus lexicon*: every word type that occurs in the text the language model is estimated on, that is in the IAM training lines and in the Brown corpus, 57 382 types.

Measured on the test transcriptions, these cover 84.8%, 93.9% and 96.4% of the test tokens. The corpus lexicon is four times smaller than the word list and yet covers more, because it is built from running text: it contains proper nouns (*Evans, Malcolm, BBC*), inflected and possessive forms, and each of them in the case in which people write it, whereas the word list is largely lower-case uninflected entries and spends most of its 239 K types on words that no one in IAM wrote. What is left uncovered in either case is proper nouns absent from the corpus, rare inflections and punctuation-bearing strings. CRNN-LX uses the corpus lexicon; that choice, like every other in this section, was made on the validation partition (Section 5.2). No lexicon is built from test transcriptions.

Language model. Two models are compared. The *unigram frequency prior* ignores the context: it is the add-one-smoothed relative frequency of c among the 47 999 tokens of the training transcriptions, with $\alpha = 5$. It is the corrector of an earlier version of this work and serves as the reference. The *trigram with line context*, used by CRNN-LX, is an interpolated Kneser–Ney trigram² with absolute discount 0.75 and an additive floor over the model’s vocabulary — the lexicon in use together with the types of the corpus, which is 57 382 in the proposed configuration and 271 303 with the word-list lexicon — so that every candidate keeps a positive probability; an unseen trigram context degrades to the bigram and then to the unigram estimate. It is estimated on the same 47 999 tokens, read as text lines, and, in the proposed configuration, additionally on the Brown corpus⁶ (57 340 sentences, 1.16 M tokens). Both external resources are taken from the Natural Language Toolkit distribution — the word list of the word-list lexicon from its **words** corpus and the text from its **brown** corpus — so that either can be reproduced with one download. Tokens are case-sensitive and punctuation marks are kept as tokens, matching the IAM transcriptions. In the proposed configuration the lexicon and the language model therefore come from the same text: the corpus supplies both the words that may be produced and the statistics that rank them.

Context. IAM word crops are cut from text lines and keep their position on the line, so the recognizer’s outputs for the neighbouring crops are available at test time. The context in Eq. (1) is the system’s *own output* for the preceding word crops of the same line — and, in the whole-line variant below, for the following ones as well — never their reference transcription. A line is a contiguous run of crops; the run is broken, and the context reset, wherever a crop is missing from

the split (crops whose segmentation IAM flags as not `ok`). On the test set 88.5% of the words have a preceding crop. The optical model still sees one isolated crop at a time; only the post-corrector reads across the line, and the no-context figure is reported alongside for the setting in which words are genuinely isolated.

Decoding the line. Two decoders are compared. The *left-to-right* decoder, used by CRNN-LX, corrects the words of a line in reading order, scoring each by Eq. (1) with h_1h_2 set to its own two previous outputs, so a word is conditioned on its left neighbours only. The *whole-line* decoder instead chooses the word sequence $w_1 \dots w_n$ of the line that maximizes $\sum_i [\log P(w_i | w_{i-2}, w_{i-1}) - \alpha d_i]$ over all combinations of candidates, by exact second-order Viterbi search; since w_i also enters the terms of w_{i+1} and w_{i+2} , every choice is conditioned on the words to its left *and* right. Each out-of-lexicon hypothesis contributes a column of its candidates (at most the ten with the best context-free score); an in-lexicon hypothesis contributes only itself. Two options of the whole-line decoder are tested and selected on validation: whether an out-of-lexicon hypothesis may also stay as it is, as a candidate at distance zero with the model’s unseen-word floor probability, so that a correct word absent from the lexicon (a proper noun, an inflection) survives when the line supports it (*keep-OOV*); and whether an in-lexicon hypothesis may be replaced by a lexicon neighbour at distance one, so that real-word errors such as *form/from* become correctable (*real-word*). For the whole-line decoder α is selected from $\{1, 2, 3, 4, 5, 7, 10, 15, 20, 30\}$, a wider grid than the $\{3, 5, 7, 10\}$ of the left-to-right decoder because the line objective shifts the useful range. Every selection — the lexicon, α , the options and the decoder itself — is made on the validation writers and applied once to the test set. Which decoder wins depends on the lexicon (Section 5.2), and the whole-line decoder’s two options are not equally important; Section 5.2 scores them separately.

External corpus. The IAM prompts were taken from the LOB corpus; Brown is a different corpus of the same design (American English, 1961), so overlap has to be measured rather than assumed. Short sequences are shared, as any two samples of English are: 21.4% of the word trigrams on the test lines and 4.3% of the 4-grams occur in Brown. Longer ones are not: of the 5-grams 0.63%, of the 6-grams 0.11%, of the 8-grams none; the longest shared sequence is seven words and the shared strings are idioms (*in spite of the fact that, on the other hand*). The same check against the IAM training lines gives 5.3% of the trigrams, 0.06% of the 5-grams and none of the 6-grams. The corpus therefore supplies English word statistics, not the test text, and the corrector never sees an IAM test transcription. The corpus lexicon inherits that property: it is the vocabulary of this text, so it contains words that IAM test pages also use — which is what any lexicon is for — but no entry is taken from a test transcription.

4. Experimental Setup

4.1. Evaluation Metrics

Let $\mathcal{T} = \{(\hat{y}_i, y_i)\}_{i=1}^N$ be the set of $N = 20\,310$ predicted/reference word pairs. *Word accuracy* (WA) is the exact-match rate

$$\text{WA} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i], \quad (2)$$

and the word error rate (WER) follows as $\text{WER} = 1 - \text{WA}$. The *character error rate* (CER) aggregates Levenshtein distances $d(\cdot, \cdot)$ over the corpus,

$$\text{CER} = \frac{\sum_{i=1}^N d(\hat{y}_i, y_i)}{\sum_{i=1}^N |y_i|}, \quad (3)$$

so that a single badly recognized long word cannot dominate the score.

4.2. Confidence Intervals

Because WA is a proportion estimated on a finite test set, we report Wilson 95% confidence intervals (CIs)²⁵ rather than normal-approximation intervals, which are unreliable near the boundaries. For k correct predictions out of N and $z = 1.96$,

$$\text{CI}_{95} = \frac{\hat{p} + \frac{z^2}{2N} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}, \quad \hat{p} = \frac{k}{N}. \quad (4)$$

With $N = 20\,310$ the half-width of the interval is 0.5–0.6 pp over the accuracies reported in this paper: 0.51 pp at the accuracy of CRNN-LX and 0.60 pp at the lowest figure we report.

4.3. Significance Testing

Two systems evaluated on the same test set produce paired outcomes, so we use the exact McNemar test. Let b be the number of samples that the first system recognizes correctly and the second does not, and c the converse. Under the null hypothesis the discordant pairs are distributed $\text{Bin}(b+c, \frac{1}{2})$, giving the two-sided p -value

$$p = \min \left(1, 2 \sum_{i=0}^{\min(b,c)} \binom{b+c}{i} 2^{-(b+c)} \right). \quad (5)$$

We treat $p < 0.01$ as significant.

4.4. Reproducibility

All six configurations were trained on the same machine (one NVIDIA GeForce RTX 4070 Laptop GPU with 8 GB, PyTorch 2.7.1 with CUDA 11.8, Python 3.13)

from the same code revision, with seed 42, at a median of 42–58 s per epoch (single epochs up to 179 s); one configuration takes between 0.9 and 1.5 hours and the six together 7.1 hours. The two endpoint configurations, CRNN-B and CRNN-LX, were additionally re-trained with seeds 123 and 456 by one of the authors on a different machine (one NVIDIA T4 through a hosted notebook service), from the same code revision, the same split files — verified byte-identical apart from line endings — and the same hyperparameters, including the corrected elastic amplitude of Section 3.4, which we checked in the run logs of those trainings; only the seed and the hardware differ. The four intermediate configurations were trained once, and differences smaller than the confidence interval are not interpreted. Every number reported below comes from evaluation passes in single precision with cuDNN restricted to its deterministic algorithms, in which the six optical models and all post-corrections are scored against identical greedy hypotheses, model by model; two separate processes produce byte-identical hypotheses for all 20 310 test words. Without that restriction single precision was reproducible within a process but one word changed between processes, and mixed-precision inference shifted 2 words between runs (0.01 pp). The post-corrector is deterministic: candidates are enumerated in a fixed order (shorter first, then lexicographic), ties keep the first candidate, and the output does not depend on the order in which lines are processed. Two numerical self-tests, distributed with the code, confirm that the trigram’s conditional distributions sum to one over the model’s vocabulary for seen and unseen contexts — checked in its largest configuration, 271 303 types — and that the whole-line decoder returns the same optimum as an exhaustive search over all candidate combinations on 400 random line layouts. Code, split definitions, the split-verification script of Section 3.1, the language-model and leakage-check scripts, per-word predictions and evaluation scripts are available at <https://github.com/Ridvan013/CRNN-Handwriting-Recognition> (branch `feature/aachen-v3-extended-trigram`); the trained weights exceed the repository file-size limit and are distributed separately.

5. Results

5.1. Augmentation Ablation

Table 2 is the central result, and it separates into two layers.

The conventional pipeline matters. Removing it costs 2.24 pp (83.41 to 81.17%) and 1.55 pp of character error rate; 1 302 words are recognized only by the baseline against 848 only by the anchor, and the exact McNemar test rejects equality at $p = 1 \times 10^{-22}$. Without a lexicon the same comparison is wider still, 78.77 against 74.33% (−4.44 pp), so part of the weaker optical model is masked by the post-corrector.

The proposed transforms do not provide a statistically detectable improvement. The five configurations that keep the conventional pipeline reach 83.41–83.89%, and none differs from the baseline at the $p < 0.01$ threshold of Section 4. The fully augmented scheme we propose is not even the nominal best: +elastic reaches

83.89% against 83.80% for CRNN-LX, a difference of 0.09 pp, or 18 of the 20 310 words. Against the baseline, CRNN-LX is 0.39 pp higher at $p = 0.049$, +elastic 0.48 pp at $p = 0.016$, morphological perturbation 0.26 pp at $p = 0.193$ and the widened photometric range 0.21 pp at $p = 0.305$; the two largest would pass a 0.05 threshold but not the 0.01 used here, nor a correction for the five comparisons made against the baseline. The ordering also moves with the post-corrector, which is why we report the same pair of optical models under four of them: the unigram prior on the word-list lexicon gives +0.37 pp at $p = 0.063$, the trigram on that lexicon +0.44 pp at $p = 0.025$, its whole-line variant +0.32 pp at $p = 0.132$, and the corrector of Table 2 0.39 pp at $p = 0.049$. The sign never changes and the threshold is never crossed, but which augmented configuration comes out on top does change.

Three seeds resolve the question, and they resolve it against the transforms. Table 3 repeats CRNN-B and CRNN-LX with seeds 123 and 456. The difference between the two changes sign: CRNN-LX is 0.39 pp above CRNN-B at seed 42, 0.45 pp *below* it at seed 123 and 0.66 pp above it at seed 456. All three per-seed comparisons fall under the conventional 0.05 threshold — at $p = 0.049$, 0.026 and 1×10^{-3} — yet two of them point in opposite directions: seed 123 favours the baseline, seed 456 the transforms. Averaged over the three seeds the difference is $+0.20 \pm 0.58$ pp (paired $t(2) = 0.60$, $p = 0.61$), and one unchanged configuration spans 0.8 pp on its own (CRNN-LX: 83.44–84.21%). The run-to-run spread is therefore two to four times the effect under study, which is why we report the augmentation comparison as unresolved rather than as a small gain; the conventional pipeline, at 2.24 pp, sits an order of magnitude above that noise floor and is untouched by the argument. A practical consequence is that a single-seed McNemar test on 20 K words can report a significant augmentation effect of either sign, so the per-configuration p -values of Table 2 should be read as descriptive. Because the configurations are nested, Table 2 reports two differences: Δ_B against the baseline CRNN-B, and Δ_P against the +wide photometric configuration on which elastic and morphological perturbation are built, which isolates each transform. By Δ_P , elastic perturbation adds 0.28 pp, morphological perturbation 0.05 pp and the two together 0.19 pp; the widened photometric range itself adds 0.21 pp, which is why Δ_B is larger by that amount (+0.48, +0.26 and +0.39 pp). On the per-word level these five disagree on 3 315 words (16.3%), yet the disagreements largely cancel: for the baseline–CRNN-LX pair, 769 words are recognized only by the baseline and 849 only by CRNN-LX. Among configurations that already augment, the choice of transform changes *which* words are recognized far more than *how many*.

Fig. 3 shows the mechanism. The anchor separates from the rest early: by epoch 53 its training loss is down to 0.018, where the augmented runs are still at 0.056–0.070, while its validation accuracy peaks at 82.57% at epoch 38 and early stopping ends the run at epoch 53. It fits the training set far more tightly and generalizes worse: its validation loss ends at 0.46 against 0.32–0.37 for the augmented runs. Those five keep improving to epoch 70–87 and converge to a plateau of 84.3–84.8%

Table 2. Augmentation ablation on the Aachen writer-disjoint test set ($N = 20310$). All rows share the network, schedule and post-corrector (the corpus lexicon of Section 3.5, Kneser–Ney trigram on IAM+Brown, left-to-right line context; the edit penalty, selected on validation for each model separately, is $\alpha = 7$, and 10 for CRNN-LX). The nominal best row is not the proposed configuration, and the difference is 18 words; see the text. Δ_B is the difference in WA to CRNN-B and Δ_P the difference to the +wide photometric configuration on which elastic and morphological perturbation are added; both are computed from unrounded accuracies. p is the exact McNemar test against CRNN-B on per-word outcomes. The first row is the zero-augmentation anchor and acts as a positive control. Best validation WA and the epoch at which it was reached are given for reference (validation: 7 205 words, 55 writers).

Configuration	WA (%)	CER (%)	Wilson 95% CI	Δ_B	Δ_P	p	best val WA (ep.)
no augmentation	81.17	9.82	[80.63, 81.70]	-2.24	—	1×10^{-22}	82.57 (38)
CRNN-B (baseline)	83.41	8.27	[82.89, 83.91]	(ref.)	—	(ref.)	84.61 (87)
+ wide photometric	83.61	8.20	[83.10, 84.12]	+0.21	(ref.)	0.305	84.34 (76)
+ elastic	83.89	7.97	[83.38, 84.39]	+0.48	+0.28	0.016	84.69 (77)
+ morphological	83.66	8.12	[83.15, 84.17]	+0.26	+0.05	0.193	84.58 (84)
CRNN-LX (all)	83.80	7.96	[83.29, 84.30]	+0.39	+0.19	0.049	84.77 (70)

Table 3. Seed repeats of the two endpoint configurations, scored with the corrector of Table 2 ($N = 20310$). Seed 42 is the run reported throughout the paper; seeds 123 and 456 differ from it only in the seed and in the machine they were trained on (Section 4). Δ is CRNN-LX minus CRNN-B within a seed and p the exact McNemar test on that pair. All three differences fall below the conventional 0.05 level — one of them below the 0.01 threshold used in this paper — and they disagree in sign.

Seed	CRNN-B WA (%)	CRNN-LX WA (%)	Δ (pp)	p	CER: CRNN-B / CRNN-LX (%)
42	83.41	83.80	+0.39	0.049	8.27 / 7.96
123	83.89	83.44	-0.45	0.026	7.96 / 8.34
456	83.55	84.21	+0.66	1×10^{-3}	8.30 / 7.72
mean $\pm$ SD	83.62 $\pm$ 0.25	83.82 $\pm$ 0.38	+0.20 $\pm$ 0.58	0.610 ^a	8.18 $\pm$ 0.19 / 8.01 $\pm$ 0.31

^aPaired t test on the three per-seed differences, $t(2) = 0.60$; the p values above it are per-seed McNemar tests.

validation accuracy, and among them the curves are indistinguishable after the first fifteen epochs; their training losses differ a little (the fully augmented model fits the training set least tightly, as a regularizer should), but the spread that reaches validation accuracy is only 0.4 pp. With 47 997 training words from 283 writers, the conventional pipeline already covers the variability that the proposed transforms simulate.

5.2. Lexicon Ablation

Table 4 holds the optical model fixed (CRNN-LX, best checkpoint) and varies only what happens after greedy decoding. The effect is an order of magnitude larger than that of the augmentation. Table 4 separates the two things a lexical corrector consists of: *which* words it may produce (the lexicon, down the rows) and *how* it chooses among them (the ranking and the line decoding, across the columns). Reading across a row isolates the ranking at a fixed lexicon; reading down a column

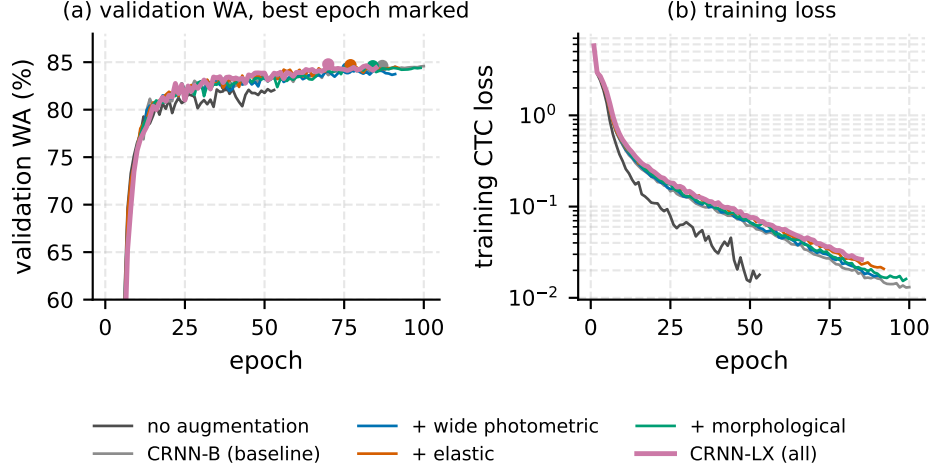


Fig. 3. Training dynamics of the six configurations: (a) validation word accuracy with the best epoch marked, (b) training CTC loss. The zero-augmentation anchor drops fastest in training loss, stops earliest and reaches the lowest validation accuracy; the five augmented curves coincide.

isolates the lexicon at a fixed ranking. All twelve cells correct the same greedy hypotheses of the same checkpoint.

Composition decides, not size (Fig. 4). Reading Table 4 down a column: with the same corrector, the corpus lexicon beats the word-list lexicon by 2.14 pp (81.66 to 83.80%) although it holds four times fewer types, and it does so in every column. The reason is coverage of the words people actually wrote: 96.4% against 93.9%, because a lexicon read off running text contains the proper nouns, the inflected and possessive forms and the capitalizations that a dictionary list omits, and spends none of its entries on words that never occur. Coverage, in turn, is what the corrector can reach at all: the 3.6% of test tokens outside the corpus lexicon can only be got right by leaving them alone.

The sign of the correction is set by whether replacement is forced, not by the size of the lexicon. Against the training lexicon, replacing every unlisted hypothesis costs accuracy: edit distance alone gives 76.42% and the unigram prior 77.04%, both below the 78.83% of lexicon-free decoding, because a correctly recognized word that the 7K list happens not to contain is replaced by a wrong one. The same 7K lexicon becomes useful as soon as the decoder may decline to replace: the whole-line decoder with keep-OOV reaches 81.32%, i.e. 2.49 pp *above* lexicon-free decoding with a lexicon that covers only 84.8% of the test tokens. What hurts is not a small vocabulary but the obligation to use it.

Ranking adds about a point and context almost as much. Within a lexicon, moving from edit distance alone to the unigram prior is worth 0.63–1.23 pp, and

replacing the prior by the trigram with line context a further 0.52–0.93 pp. On the corpus lexicon the three steps decompose the corrector’s 4.97 pp as +2.89 for the lexicon and its edit bound, +1.19 for the frequency prior and +0.89 for the trigram and the line context; on the word-list lexicon the same three steps give +0.67, +1.23 and +0.93. The ranking terms are almost the same in both; the lexicon is where the two columns part company.

Which decoder is best depends on the lexicon, and the whole-line column of Table 4 changes two things at once: the line is decoded jointly *and* an out-of-lexicon hypothesis may be kept. Scored separately, the two are very unequal. Joint decoding is worth about the same whatever the lexicon: +0.12 pp on the training lexicon and +0.19 pp on the corpus lexicon. The freedom not to replace is what depends on it, and it changes sign: +3.65 pp on the training lexicon, where 15.2% of the reference tokens are missing and a correct word would otherwise be overwritten, but −1.22 pp on the corpus lexicon, where 96.4% are present and the same freedom mostly preserves errors. The two net effects are the +3.77 and −1.03 pp of Table 4, and the mechanism behind them is the freedom, not the joint decoding.

CRNN-LX is the corpus lexicon with the left-to-right trigram: 83.80% word accuracy and 7.96% character error rate. One variant we did not include in the selection is worth reporting. The whole-line decoder with forced replacement — joint decoding without keep-OOV — reaches 83.99% and 7.90% on the test partition, and is 0.21 pp above the left-to-right decoder on the validation partition. Both margins are inside the confidence interval of the partition they are measured on (0.51 pp on test, 0.77 pp on validation), so we report the simpler decoder rather than the nominally better one, and we do not claim that these two decoders can be separated at this sample size. The difference between them is a tenth of what the lexicon is worth.

Per word, that corrector changes 2 724 of the 20 310 hypotheses (13.4%), repairs 1 216 and breaks 207. The corrector of the word-list lexicon changed 2 939, repaired 1 076 and broke 501: the corpus lexicon repairs more *and* breaks less than half as often, which is the whole of its advantage.

Table 4. Post-correction with the optical model fixed (CRNN-LX checkpoint, $N = 20\,310$). Every cell applies a different corrector to the *same* greedy hypotheses and reports WA (CER) in per cent; the Wilson half-width runs from 0.51 pp in the best cell to 0.58 pp in the worst. Columns: the candidate within the edit bound is chosen by distance alone, by the add-one unigram prior, by the Kneser–Ney trigram with the system’s own left context, or by the whole-line decoder with keep-OOV of Section 3.5. Rows: the three lexicons of Section 3.5 with their coverage of the test tokens. Lexicon-free greedy decoding (CRNN-G) gives 78.83 (8.60). The edit penalty is $\alpha = 5$ for the unigram prior and selected on validation otherwise; CRNN-LX is the bold cell.

Lexicon	Types	Coverage	edit only	+ unigram	+ KN3, L→R	+ KN3, line
training	7 173	84.8%	76.42 (11.19)	77.04 (10.94)	77.56 (10.69)	81.32 (8.34)
word list	239 126	93.9%	79.51 (9.71)	80.74 (9.09)	81.66 (8.58)	81.95 (8.54)
corpus	57 382	96.4%	81.72 (8.95)	82.91 (8.43)	83.80 (7.96)	82.77 (8.33)

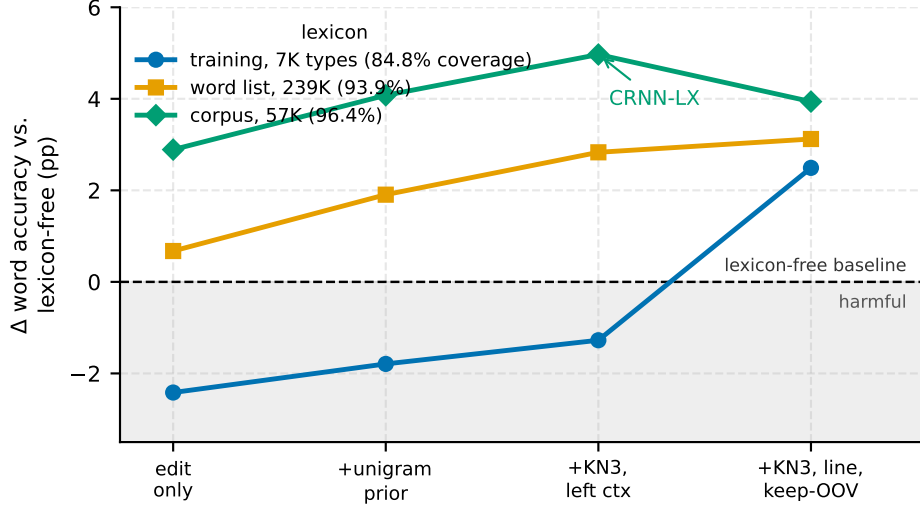


Fig. 4. What the post-corrector contributes, against lexicon-free decoding, for the CRNN-LX optical model. Each line is one lexicon; the four points are the correctors of Table 4. The lexicon separates the lines by more than the correctors separate the points along them: the corpus lexicon (57K types, 96.4% coverage) is above the word list (239K, 93.9%) everywhere, and the training lexicon (7K, 84.8%) is below lexicon-free decoding until the decoder is allowed to keep an out-of-lexicon hypothesis.

The character error rate follows the same ordering, once replacement is not forced. A whole-word corrector can only replace a hypothesis by a complete lexicon entry; when the replacement is wrong, a near-miss with one character error becomes a different word with several. That is why the training lexicon with forced replacement pays 11.19% against 8.60% for lexicon-free decoding, and why the word-list lexicon with the unigram prior still pays 9.09% while gaining word accuracy. With the trigram the word-list lexicon ends level on character error rate (8.58% against 8.60%), and only the corpus lexicon improves both by a clear margin: 7.96% against 8.60%, with 4.97 pp of word accuracy gained. Word accuracy and character error rate therefore have to be reported together whenever a whole-word corrector is used; a configuration can buy one with the other, and for most of Table 4 it does.

5.3. Lexical Decoding Instead of Post-Hoc Correction

A lexicon can also be imposed while decoding rather than afterwards. Word beam search (WBS)²⁰ restricts CTC beam search to dictionary words, and unlike a post-corrector it sees the whole output distribution instead of the single string greedy decoding returns. Table 5 runs the authors’ own implementation on the output distributions of the same optical model, with the same three lexicons.

The pattern of Table 4 repeats, more sharply. With the training lexicon WBS reaches only 76.02%, below lexicon-free decoding, because every output is forced into a 7K-word dictionary; its character error rate, 14.87%, is the worst figure in either table. With the word-list lexicon it reaches 81.26% (81.70% once the dictionary also holds the capitalized forms), and with the corpus lexicon 84.13%. Changing the lexicon is therefore worth 2.4 pp to WBS, whereas its language model is worth nothing: adding the word bigram model leaves word accuracy at 84.07% and raises the character error rate. The case variants that help the other two dictionaries do not help this one — they lower it to 83.90% — because a vocabulary read off running text already holds each word in the case in which it is written, and the extra forms only add confusable entries. That is the same property, seen from the decoder’s side, that makes the corpus lexicon worth 2.14 pp to our post-corrector.

Against our post-corrector the comparison is close. At the matched corpus lexicon WBS is 0.32 pp above CRNN-LX in word accuracy (84.13 against 83.80%) and 0.27 pp behind it in character error rate, with 316 words recognized only by CRNN-LX and 382 only by WBS; the exact McNemar test gives $p = 0.014$, above the threshold of Section 4. The two fail differently, though. WBS reaches words that lie outside the edit bound of a post-corrector — *Seplemben* → *September*, *playmright* → *playwright*, *accounttant* → *accountant* — because it never commits to the greedy string. Our corrector leaves a hypothesis alone unless a lexicon entry lies within two edits, which is why its character error rate stays lower. Neither can produce a word the lexicon does not contain, and that is what sets the ceiling for both.

Table 5. Word beam search on the *same* optical model’s output distributions (the authors’ implementation,²⁰ beam width 25, smoothing 0.01), against the post-corrector of Table 4. WBS emits dictionary entries verbatim, so the “+ case” rows also hold the lower-case and capitalized form of every entry, which our membership test accepts anyway. “NGrams” additionally uses the word bigram model WBS estimates from the corpus text. The last row is CRNN-LX.

Decoder	Dictionary	Types	WA (%)	CER (%)
WBS, Words	training lexicon	7 173	76.02	14.87
WBS, Words	training lexicon + case	13 281	76.48	14.28
WBS, Words	word-list lexicon	239 126	81.26	9.69
WBS, Words	word-list lexicon + case	473 717	81.70	9.24
WBS, Words	corpus lexicon	57 382	84.13	8.23
WBS, Words	corpus lexicon + case	100 761	83.90	8.31
WBS, NGrams	corpus text (bigram LM)	57 382	84.07	8.64
CRNN-LX (post-correction)	corpus lexicon	57 382	83.80	7.96

5.4. Comparison with Prior Word-Level Systems

Table 6 places CRNN-LX next to seven published word-level systems and states, for each, whether a lexicon or language model was applied at test time. Four of

our own rows are given, because they answer different questions: CRNN-G is the optical model alone; the *no-context* row applies the corpus lexicon with the unigram prior and is the figure for words that are genuinely isolated; CRNN-LX additionally reads the recognizer’s own outputs for the preceding crops of the line, which none of the listed systems does; and the last row imposes the same lexicon inside the decoder instead (Section 5.3). The upper block lists results obtained on the Aachen writer-disjoint partition. The closest match in protocol is the sequence-to-sequence recognizer of Sueiras et al.:²³ the sizes of their partition (47 952, 20 306 and 7 558 words, as compiled in¹⁶) agree to within a few dozen words with the ok-filtered Aachen training, test and validation lists (47 999, 20 310 and 7 559) from which ours are drawn, and they report lexicon-free word and character error rates on the full test partition exactly as we do. Under lexicon-free decoding CRNN-G is 2.6 pp more accurate (78.83% against 76.20%) at a slightly lower CER (8.60% against 8.80%); with the corpus lexicon the margin is 7.6 pp (6.7 pp without line context). CRNN-LX is 0.29 pp below the candidate-fusion model of Kang et al.⁹ and 0.80 pp below AttentionHTR,¹¹ which additionally transfers knowledge from 14.4 M scene-text images, and 1.25 pp *above* the attention model of Kang et al.;¹⁰ the same optical model decoded with word beam search (Section 5.3) reaches 84.13%, level with Kang et al.⁹ At character level the gap remains (5.79–6.88% against our 7.96–8.60%), consistent with the autoregressive decoders of those systems modeling inter-character dependencies that a CTC output with whole-word post-correction cannot. The lower block reports three systems measured on other partitions of IAM; CRNN-LX is above all three and CRNN-G above two, but the partitions differ in writer overlap and size and we draw no conclusion from margins of this size. The row taken from Dutta et al.⁵ is HWRCNet’s re-training of their network; their own figure for it, a 12.61% word error rate, is measured without punctuation or letter case and after synthetic pre-training and test-time augmentation, so it is not on the same scale as the case-sensitive accuracies listed here. Our rows in this table are there to situate the system, not to compete with it: the comparisons this paper controls are those of Tables 2–5, in which one ingredient changes at a time — the augmentation in Tables 2 and 3, and the lexicon, the ranking rule or the place it is applied in Tables 4 and 5, with the optical model held fixed and, in Table 4, the same greedy hypotheses throughout.

5.5. Error Analysis

CRNN-LX misrecognizes 3 290 of the 20 310 test words, 16.2%. Only 293 of them (8.9%) differ from the reference in letter case alone; the remaining errors are genuine. At character level the aligned errors consist of 4 800 substitutions, 1 279 deletions and 441 insertions, so the model drops characters far more often than it hallucinates them. Fig. 5 shows the ten most frequent *directed* substitutions. Adding the two directions of each pair, the most frequent confusions are between visually similar lower-case glyphs — $a \leftrightarrow o$ (215), $r \leftrightarrow s$ (139), $l \leftrightarrow t$ (95), $n \leftrightarrow r$ (83), $m \leftrightarrow n$ (80), $a \leftrightarrow e$

Table 6. Word-level results on IAM. WA in parentheses is derived as $1 - \text{WER}$ from the cited paper. Lexicon: lexical constraint at test time (none = unconstrained decoding; LM = language model without a closed lexicon; n/s = not stated). Only CRNN-LX uses the recognizer’s outputs for the preceding crops of the line; every other row recognizes each word in isolation. Bold marks the best WA and the best CER in the table.

System	Year	Lexicon	WA (%)	CER (%)
<i>Aachen writer-disjoint partition (≈ 20.3 K test words)</i>				
Sueiras et al. ²³	2018	none	(76.20)	8.80
Kang et al. ¹⁰	2018	none	(82.55)	6.88
Kang et al. ⁹	2021	LM	(84.09)	5.79
Kass and Vats ¹¹	2022	none	(84.60)	6.50
CRNN-G (ours)	2026	none	78.83	8.60
CRNN-LX, no context (ours)	2026	57 K corpus + unigram prior	82.91	8.43
CRNN-LX (ours)	2026	57 K corpus + trigram, line context	83.80	7.96
CRNN-G + WBS ²⁰ (ours)	2026	57 K corpus, in decoding	84.13	8.23
<i>Other partitions of IAM</i>				
Mondal et al. ^{16a}	2022	none	(70.79)	9.53
CNN-RNN of, ⁵ re-trained in ^{18b}	2022	n/s	(77.14)	11.08
Rajesh et al. ^{18b}	2022	n/s	80.08	9.89

^a30 K randomly selected test words; 1 200 training images.

^bRandom 95:5 split, writer-dependent, ≈ 5.8 K test words.

(77) — so the same shapes are confused in both directions. The figure counts each direction separately, so a pair may appear in it twice, once, or — like $\mathbf{m}/\mathbf{n}$, with 37 and 43 — not at all, the tenth entry standing at 44. Case substitutions account for 8.2% of all substitutions. The confusions themselves are errors of shape, not of vocabulary: a whole-word lexicon can repair them only when the mis-shaped word falls outside the lexicon and its correct form lies within two edits, which is exactly the population that the corpus lexicon recovers in Table 4.

6. Discussion

6.1. Interpretation of the Augmentation Results

The anchor rules out the explanation that would otherwise be hardest to exclude. A null result can always mean that the experiment is too noisy to see anything, and the seed repeats confirm that the noise is real: an unchanged configuration moves by up to 0.8 pp from seed to seed, and the sign of the comparison moves with it. But the same test, the same data and the same protocol detect the removal of the conventional pipeline at $p = 1 \times 10^{-22}$, 2.2 points of word accuracy. The apparatus is not blind to augmentation; it cannot resolve *these* transforms on *this* training set. Their largest single difference (0.48 pp for elastic deformation, $p = 0.016$) does not survive the threshold, the ordering moves with the post-corrector, and the proposed-minus-baseline difference (0.39 pp, $p = 0.049$) changes sign with the seed (Table 3). What we observe is consistent with saturation, and three further observations indicate where it occurs.

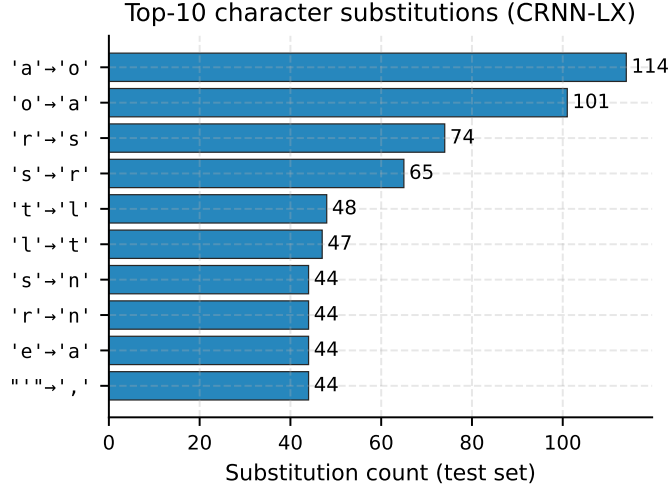


Fig. 5. Ten most frequent character substitutions of CRNN-LX on the test set (true → predicted).

First, the anchor overfits earliest and hardest: at epoch 53, where it stops, its training loss is 0.018 against 0.056–0.070 for the augmented runs, and its validation loss has risen 0.10 above its own minimum, against 0.03–0.07 for them; the augmented runs stop 32 to 47 epochs later (Fig. 3). The first increment of augmentation buys regularization that the network genuinely needs. Second, among the augmented configurations the validation curves coincide and the extra transforms neither slow convergence nor raise the plateau, while the fully augmented model reaches a slightly higher training loss — so they do regularize further, but the baseline is no longer overfitting in a way that costs accuracy on unseen writers. Third, the per-word disagreements between those configurations are large yet balanced. Together these indicate that with 48 K training words from 283 writers, the writer-to-writer variation already present in the data exceeds what elastic and morphological perturbation of 64-pixel-high crops can add on top of affine and photometric jitter. Reports of sizeable gains from such augmentation, including our own preliminary experiments, came from smaller training sets or from comparisons in which augmentation was not the only difference. The result does not imply that augmentation is useless for HTR in general — Wigington et al.²⁴ report gains on lines with a different transform family, and our own anchor shows the first increment is worth 2.2 pp — but it does show that further transforms must be measured under controlled conditions before being claimed.

6.2. Which Lexicon, and Where to Apply It

Three things are usually reported as one when a paper says it “added a language model”: the lexicon, the rule that ranks candidates inside it, and whether the lexicon is imposed during decoding or afterwards. Table 4 and Table 5 separate them, and they are not equally important.

The lexicon dominates. Holding the corrector fixed, the corpus lexicon is worth 2.14 pp over the word-list lexicon — one choice worth as much as the ranking rule and the line context contribute together (2.08 pp on the corpus lexicon, 2.16 pp on the word list), and more than either of them alone. It is also the smaller of the two, so what matters is not how many words a lexicon holds but whether it holds the ones people write: proper nouns, inflections, possessives and the capitalization they appear in. A corpus supplies those by construction; a dictionary list does not, and pays for its size with confusable neighbours — at 79.51% for edit distance alone it is 2.2 pp behind the corpus lexicon under the same rule. The practical consequence is that the coverage of the lexicon on the target vocabulary is the first number to report about a lexicon-assisted result, ahead of its size.

Whether replacement is forced matters next. Our corrector replaces any hypothesis outside the lexicon, and with the 7 K training lexicon that is harmful: 77.04% against 78.83% for no correction at all, because a correct word that the list does not contain is replaced by a wrong one. The whole-line decoder may decline to replace, and with the same lexicon it gains 2.49 pp instead. The earlier reading of these numbers — that a small lexicon is harmful — was too quick: what is harmful is the obligation to use it.

The ranking rule does what a ranking rule can. Within a lexicon, replacing edit distance by the unigram prior is worth about a point, and replacing the prior by a trigram with line context nearly another; the trigram earns that only because the corpus is large enough to have seen the contexts (on IAM text alone the same context was worth 0.10 pp). Neither approaches the lexicon term, and the ceiling is close: the corrector only ever touches the 15.2% of hypotheses that fall outside the lexicon, and feeding it the reference transcriptions of the neighbouring crops instead of its own outputs adds 0.06 pp — this figure and the 0.10 pp above were both measured on the word-list lexicon.

Finally, where the lexicon is applied matters less than one might expect. Word beam search sees the whole CTC distribution and can therefore reach words a post-corrector cannot, but at the matched lexicon the two end within 0.32 pp of each other in word accuracy ($p = 0.014$) and our corrector is 0.27 pp better in character error rate; WBS’s own language model contributes nothing once its dictionary comes from the corpus. A post-corrector has the practical advantage that the lexicon can be swapped without re-decoding, which is what made the comparisons in this section possible at all; a decoder has the advantage of not committing to one string. Both stand or fall with the lexicon.

A practical consequence is that word-level HTR results obtained with a closed

lexicon should state the lexicon’s coverage of the test vocabulary, the rule used to rank candidates, the corpus the ranking was learned from and whether any context was used, without which the reported accuracy is not interpretable. The remaining 3.6% of uncovered tokens (proper nouns, rare inflections, punctuation-bearing strings) bound what any whole-word corrector can achieve; addressing them requires an open-vocabulary character-level model.

6.3. *Comparison with Previous Word-Level HTR Systems*

Among the systems of Table 6, the recognizer of Sueiras et al.²³ is the one whose protocol matches ours most closely: isolated IAM words, the Aachen writer-disjoint partition drawn from the same ok-filtered word lists, and lexicon-free word and character error rates on the full test partition. It is also the only prior system for which we could verify the partition from the sizes reported for it (Section 5.4); on our side we additionally discard two unreadable images and five validation forms whose prompts recur in the test set. Architecturally the two differ in every stage: they extract features from a sliding window of image patches with a LeNet-5-style convolutional network and transcribe them with a recurrent encoder–decoder using attention, whereas CRNN-G is a single convolutional-recurrent network trained with CTC. Under lexicon-free decoding CRNN-G reaches 78.83% against their 76.20% (23.8% word error rate), a margin of 2.6 pp that is almost five times the half-width of our confidence interval, at a slightly lower character error rate (8.60% against 8.80%). Post-correction with the corpus lexicon widens the word-level margin to 6.7 pp without context and 7.6 pp with it. Sueiras et al. also report lexicon-constrained variants of their recognizer; those depend on the lexicon employed and are not compared here. We read the comparison as evidence that a plain CRNN trained from scratch on the 48 K Aachen training words is competitive with an early attention model under the same protocol, not as evidence about augmentation: the proposed transforms contribute nothing measurable to that margin (Tables 2 and 3).

HWRCNet,¹⁸ the closest system in construction (a CNN, a bidirectional LSTM and a CTC layer), reports 80.08% strict word accuracy, 3.72 pp below CRNN-LX (2.83 pp below its no-context variant) and 1.25 pp above CRNN-G. It is measured on a random 95:5 partition whose test set is writer-dependent and about 3.5 times smaller than ours, so neither difference is interpretable.

6.4. *Threats to Validity*

The augmentation differences of 0.2–0.5 pp are below what one training run can resolve: with three seeds the CRNN-B–CRNN-LX difference changes sign and averages $+0.20 \pm 0.58$ pp (Table 3), so we report the transforms as unresolved rather than as small gains. Only the two endpoint configurations were repeated; the four intermediate ones were trained once, and their individual p -values in Table 2 therefore carry no more weight than the seed-42 row of Table 3 does. The repeats also ran on different hardware, so the spread we quote includes any hardware-related

nondeterminism — which only reinforces the conclusion that an effect of this size is not resolvable here. The 2.24 pp effect of the conventional pipeline is an order of magnitude larger than that resolution limit, so the positive control does not depend on the seed in the way the null result does. Three corrector effects were measured on all six optical models, and they are not equally stable. Forced replacement against the training vocabulary costs the five augmented models 2.4–2.7 pp of word accuracy, but the zero-augmentation anchor only 1.1 pp; and the frequency prior on the word-list lexicon adds 1.1–1.3 pp to the five but 2.1 pp to the anchor. Both terms are thus larger the weaker the optical model is, which is what a lexicon should do: it has more to repair. The left-to-right trigram is the stable one — it adds 0.85–0.96 pp over that prior on every one of the six models, with $\alpha = 7$ selected on validation each time. The comparison *between* lexicons in Table 4, on the other hand, was run on one optical model, and the six re-scored models of Table 2 only confirm that the final corrector behaves consistently, not that the 2.14 pp lexicon effect has the same size everywhere. The effects are also measured with one candidate generator and one edit bound; a different bound would change their size, though the ordering of the columns is unlikely to change. The whole-line decoder has two options and a wider penalty grid, and the validation partition does not separate its variants from the left-to-right decoder: on the corpus lexicon it ranks the whole-line decoder with forced replacement 0.21 pp above our choice and the keep-OOV variant 0.75 pp below it, both inside its own 0.77 pp interval (Section 5.2). On the word-list lexicon the option validation rejected (real-word substitution) reaches almost the same test accuracy as the one it selected by a different route (81.94% against 81.95%) — evidence that the selection is not finely tuned to the test set, but again that the validation partition cannot separate such variants. The confidence intervals we report treat the test words as independent draws. They are not: they come from 161 writers, and per-writer accuracy ranges from 33% to 98% (SD 9.5 pp), so errors arrive in blocks. A bootstrap that resamples writers instead of words widens the interval of CRNN-LX from ± 0.51 to ± 1.82 pp, and one that resamples forms gives ± 1.07 pp; a bootstrap over words reproduces the Wilson interval exactly, which locates the difference in the clustering rather than in the arithmetic. The comparisons in this paper are paired on the same writers, so the writer effect largely cancels within them, but generalization to other writers is less certain than a single row of Table 2 suggests. Validation accuracy, which the training loop measures with the unigram corrector, exceeds the test accuracy of that same corrector by 4.0 to 4.7 points, in every configuration; the validation writers are used for checkpoint selection and are only 55 in number, so this gap is expected and does not indicate a fault. The external systems in Table 6 were trained and evaluated by their authors with their own preprocessing and, in the lower block, on a different partition; we therefore treat only our own runs as a controlled comparison. The corpus lexicon is the vocabulary of the IAM training lines and of the Brown corpus; it contains no IAM test transcription, but it is a closed lexicon built with one eye on the domain, and CRNN-LX should be read as a lexicon-assisted result. A lexicon assembled

from a corpus closer to the target material would raise the figure further, and one assembled from a corpus further away would lower it; the coverage we report (96.4% of the test tokens) is the number that makes such results comparable. Two further caveats concern the trigram. Its corpus is external to IAM; we measured the verbatim overlap with the test lines (Section 3.5) and found idioms only, but a corpus with the same content as the prompts would inflate the result, and any replacement corpus should be checked the same way. And its context is the recognizer’s output for the neighbouring crops, which exists because IAM word images are cut from lines; on a collection of genuinely isolated words the applicable figure is the no-context row of Table 6 (82.91%), not 83.80%. The edit penalty, the decoder and its options were selected on the validation writers and applied once to the test set.

7. Conclusion

CRNN-LX recognizes handwritten words with 83.80% accuracy (7.96% character error rate) on the complete Aachen writer-disjoint test set of IAM, with a plain CRNN trained from scratch, no external image data, and a post-corrector whose only external resource is a public text corpus. Under the same partition and lexicon-free decoding the optical model is 2.6 pp more accurate than the sequence-to-sequence recognizer of Sueiras et al.;²³ the complete system is 1.3 pp above the 2018 attention recognizer of Kang et al.¹⁰ and 0.3–0.8 pp below the two strongest published systems on this partition; treating the words as isolated, without the line context, costs 0.9 pp of that. Under controlled conditions, elastic deformation, morphological perturbation and a widened photometric range produce no statistically detectable change in these figures — over three seeds their apparent effect changes sign and averages $+0.20 \pm 0.58$ pp — while removing the conventional augmentation pipeline beneath them costs 2.24 pp, a pattern consistent with the benefit of augmentation saturating before the transforms we proposed; a widely used elastic parameterization, moreover, does not deform the image at all. The post-correction stage, by contrast, moves accuracy by five points up and nearly two and a half down, and its parts are not equally important: the lexicon is worth 2.1 pp when it is read off the language-model corpus instead of a dictionary list, the ranking rule about a point, the line context nine tenths of one, and imposing the lexicon inside the decoder rather than afterwards 0.32 pp, which is not significant at our threshold. Whether a small lexicon helps or hurts turns out to depend on whether the corrector is obliged to use it.

For word-level HTR the practical conclusions are that augmentation gains should be demonstrated, not assumed, on the full training set; that an ablation reporting no effect should include the configuration in which the effect is known to exist, and should report the run-to-run spread of an unchanged configuration, which at this scale exceeded every difference we measured between augmented configurations; that lexicon coverage, the ranking rule, its corpus and its context should be reported separately alongside lexicon-assisted accuracy; and that word accuracy

and character error rate should be given together whenever a whole-word corrector is used. These are properties of the experimental setup rather than of our particular network: any word-level system that corrects against a lexicon can be decomposed the same way, and any augmentation gain of a few tenths of a point on IAM words has to be read against the run-to-run spread measured here. Future work will test whether the augmentation becomes useful in the low-data regime, replace the whole-word corrector with a character-level open-vocabulary model to reach the 3.6% of test tokens that the corpus lexicon does not contain, and ask how a lexicon should be assembled when the target domain is known, since that term dominated everything else we measured.

Acknowledgments

We thank the maintainers of the IAM Handwriting Database and of the Aachen split.

References

1. T. Bluche and R. Messina, Gated convolutional recurrent neural networks for multi-lingual handwriting recognition, in *14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, Vol. 1 (IEEE, 2017), pp. 646–651.
2. S. F. Chen and J. Goodman, An empirical study of smoothing techniques for language modeling, *Computer Speech & Language* **13**(4) (1999) 359–394.
3. G. Crosilla, L. Klic and G. Colavizza, Benchmarking large language models for handwritten text recognition, *Journal of Documentation* (2025).
4. A. F. de Sousa Neto, B. L. D. Bezerra, A. H. Toselli and E. B. Lima, HTR-Flor++: a handwritten text recognition system based on a pipeline of optical and language models, in *Proceedings of the ACM Symposium on Document Engineering (DocEng)* (2020).
5. K. Dutta, P. Krishnan, M. Mathew and C. V. Jawahar, Improving CNN-RNN hybrid networks for handwriting recognition, in *16th International Conference on Frontiers in Handwriting Recognition (ICFHR)* (IEEE, 2018), pp. 80–85.
6. W. N. Francis and H. Kučera, Brown corpus manual: Manual of information to accompany a standard corpus of present-day edited American English, for use with digital computers, tech. rep., Department of Linguistics, Brown University (Providence, Rhode Island, 1979), Revised edition.
7. A. Graves, S. Fernández, F. Gomez and J. Schmidhuber, Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks, in *Proceedings of the 23rd International Conference on Machine Learning (ICML)* (2006), pp. 369–376.
8. A. Graves and J. Schmidhuber, Offline handwriting recognition with multidimensional recurrent neural networks, in *Advances in Neural Information Processing Systems (NeurIPS)* (2008), pp. 545–552.
9. L. Kang, P. Riba, M. Villegas, A. Fornés and M. Rusiñol, Candidate fusion: integrating language modelling into a sequence-to-sequence handwritten word recognition architecture, *Pattern Recognition* **112** (2021) p. 107790.
10. L. Kang, J. I. Toledo, P. Riba, M. Villegas, A. Fornés and M. Rusiñol, Convolv, attend and spell: an attention-based sequence-to-sequence model for handwritten word

- recognition, in *German Conference on Pattern Recognition (GCPR)*, LNCS 11269 (Springer, 2018), pp. 459–472.
11. D. Kass and E. Vats, AttentionHTR: handwritten text recognition based on attention encoder-decoder networks, in *15th IAPR International Workshop on Document Analysis Systems (DAS)*, LNCS 13237 (Springer, 2022), pp. 507–522.
 12. R. Kneser and H. Ney, Improved backing-off for M-gram language modeling, in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, Vol. 1 (1995), pp. 181–184.
 13. M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li and F. Wei, TrOCR: transformer-based optical character recognition with pre-trained models, in *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37(11) (2023), pp. 13094–13102.
 14. U.-V. Marti and H. Bunke, The IAM-database: an English sentence database for offline handwriting recognition, *International Journal on Document Analysis and Recognition* **5**(1) (2002) 39–46.
 15. J. Michael, R. Labahn, T. Grüning and J. Zöllner, Evaluating sequence-to-sequence models for handwritten text recognition, in *International Conference on Document Analysis and Recognition (ICDAR)* (IEEE, 2019), pp. 1286–1293.
 16. R. Mondal, S. Malakar, E. H. Barney Smith and R. Sarkar, Handwritten English word recognition using a deep learning based object detection architecture, *Multimedia Tools and Applications* **81**(1) (2022) 975–1000.
 17. J. Puigcerver, Are multidimensional recurrent layers really necessary for handwritten text recognition?, in *14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, Vol. 1 (IEEE, 2017), pp. 67–72.
 18. B. Rajesh, A. K. Gupta, A. Raj, M. Javed and S. R. Dubey, HWRCNet: Handwritten word recognition in JPEG compressed domain using CNN-BiLSTM network, in *International Conference on Data Analytics and Learning (DAL)* (2022).
 19. RWTH Aachen University, IAM handwriting database — Aachen writer-disjoint split. Community split distributed with the RWTH Aachen speech and language processing toolkits, (2010).
 20. H. Scheidl, S. Fiel and R. Sablatnig, Word beam search: a connectionist temporal classification decoding algorithm, in *16th International Conference on Frontiers in Handwriting Recognition (ICFHR)* (IEEE, 2018), pp. 253–258.
 21. B. Shi, X. Bai and C. Yao, An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **39**(11) (2017) 2298–2304.
 22. P. Y. Simard, D. Steinkraus and J. C. Platt, Best practices for convolutional neural networks applied to visual document analysis, in *7th International Conference on Document Analysis and Recognition (ICDAR)* (IEEE, 2003), pp. 958–963.
 23. J. Sueiras, V. Ruiz, A. Sanchez and J. F. Velez, Offline continuous handwriting recognition using sequence to sequence neural networks, *Neurocomputing* **289** (2018) 119–128.
 24. C. Wigington, S. Stewart, B. Davis, B. Barrett, B. Price and S. Cohen, Data augmentation for recognition of handwritten words and lines using a CNN-LSTM network, in *14th IAPR International Conference on Document Analysis and Recognition (ICDAR)* (IEEE, 2017), pp. 639–645.
 25. E. B. Wilson, Probable inference, the law of succession, and statistical inference, *Journal of the American Statistical Association* **22**(158) (1927) 209–212.